C. COMPUTAÇÃO
UNIPAC

CENTRO UNIVERSITÁRIO UNIPAC
SCIENTIA, LIBERTAS ET FIDES
PRESIDENTE ANTÔNIO CARLOS

unipac.br
Barbacena

Computação Gráfica

Drawing Shapes

Do ponto às formas com OpenGL

Professor:

Dr. Rodrigo Fenandes dos Santos

O que vamos aprender hoje?

O caminho de seis coordenadas até um hexágono na tela

Vértices

↓

Buffers

↓

Atributos

↓

Shaders

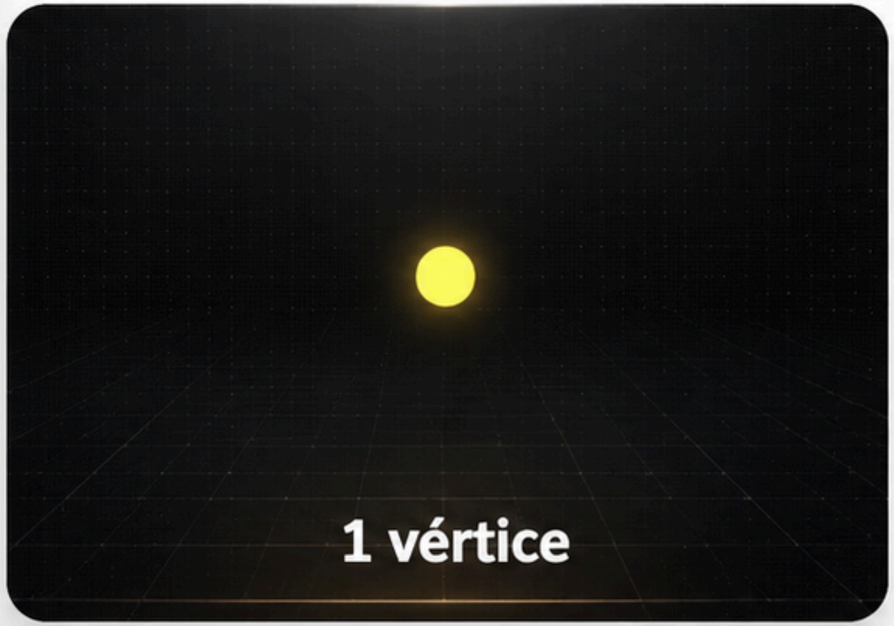
↓

Primitivas

↓

Formas

De um ponto para uma forma



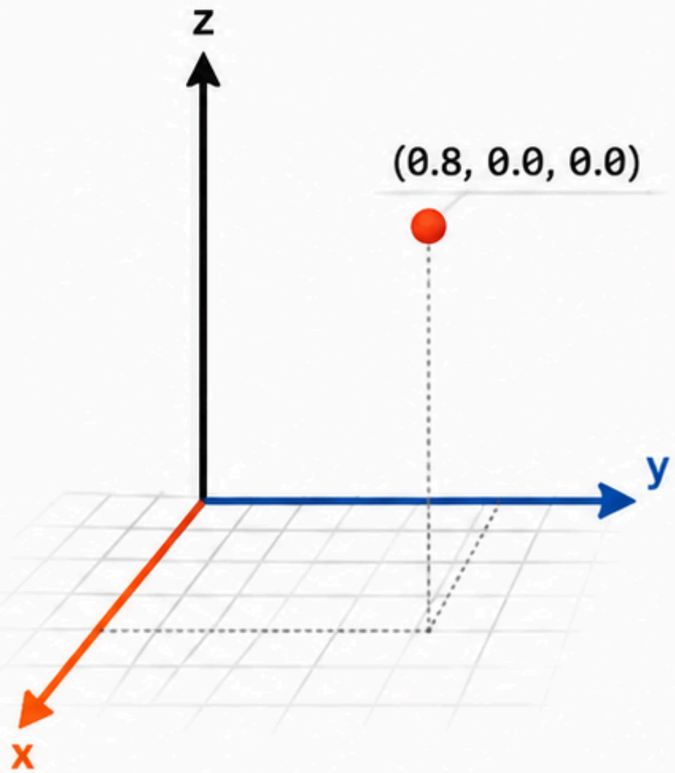
+ 5

Como o OpenGL sabe onde estão todos esses pontos?

O que é um vértice?

- Um vértice é, antes de tudo, uma posição no espaço.

➡ **vértice = posição**



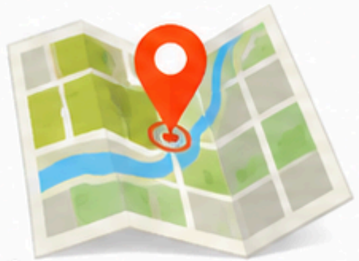
x = 0.8

y = 0.0

z = 0.0

Pense assim...

Imagine um ponto no mapa.
Você precisa informar:
"Para a direita, para cima"
e a altura" para indicar
exatamente onde ele está.
No mundo 3D é a mesma ideia!

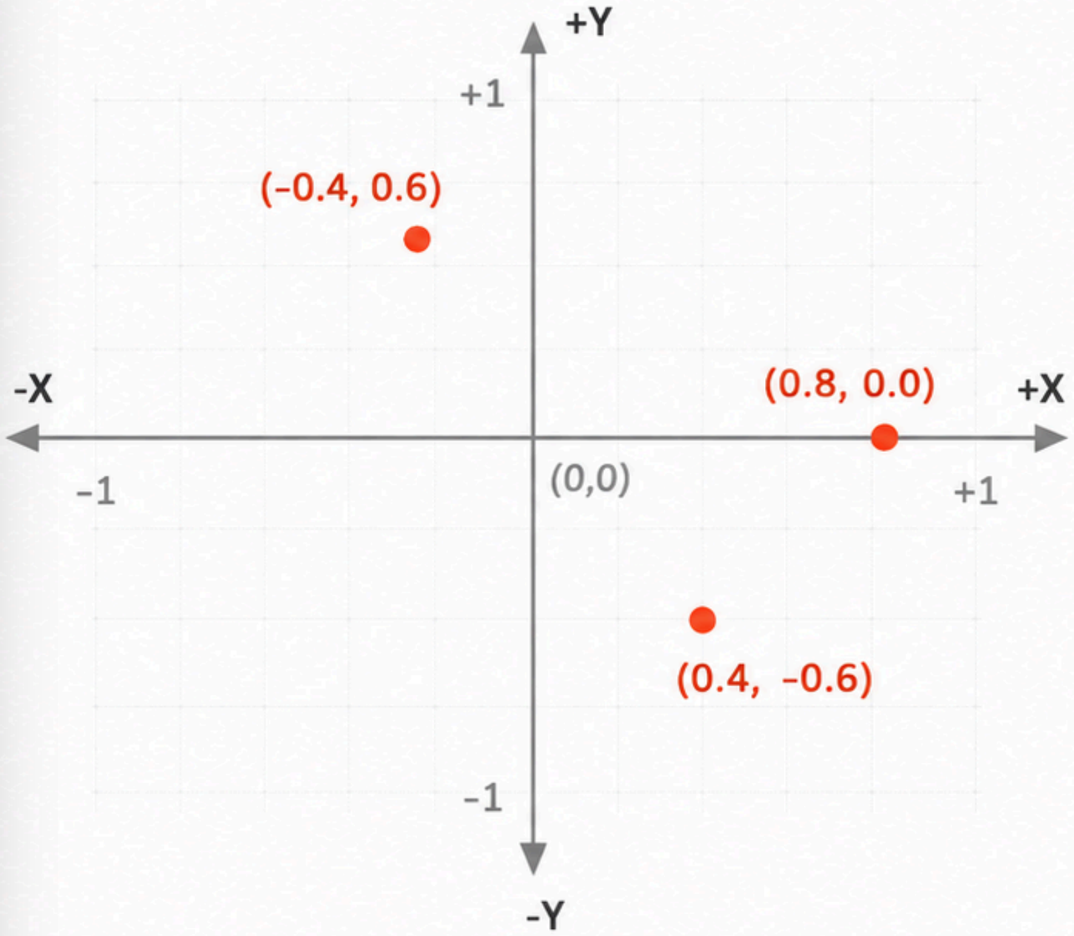


posição de um vértice

```
[0.8, 0.0, 0.0]
```

🎯 Cada vértice guarda um endereço no espaço: [x, y, z]

O sistema de coordenadas do OpenGL

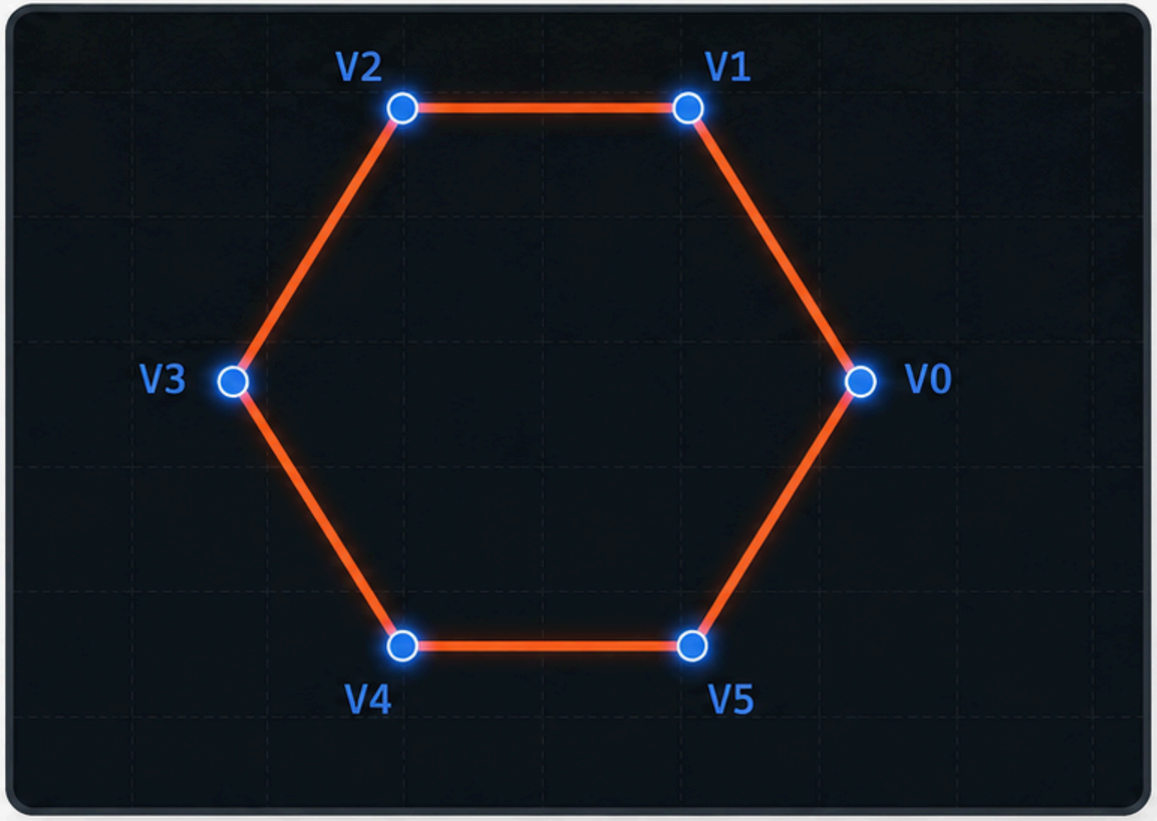


- A tela é representada por um plano que vai de -1 a +1, em x e em y.
- É o chamado sistema NDC — coordenadas normalizadas do dispositivo.
- O centro da tela é o ponto (0, 0).
- Alguns dos vértices reais do test-2-3.py estão marcados no plano ao lado.

Seis pontos formam nosso hexágono

test-2-3.py

```
1   positionData = [
2         [ 0.8,  0.0, 0.0],
3         [ 0.4,  0.6, 0.0],
4         [-0.4,  0.6, 0.0],
5         [-0.8,  0.0, 0.0],
6         [-0.4, -0.6, 0.0],
7         [ 0.4, -0.6, 0.0]
8   ]
```



Cada linha da lista representa a posição de um vértice no espaço.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

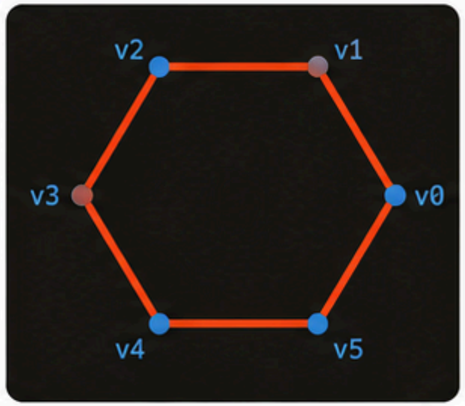
Por que usamos uma lista?

OBJETO
Precisamos representar um hexágono.

VÁRIOS VÉRTICES
Um hexágono tem seis vértices (pontos).

LISTA DE DADOS
Agrupamos todos os vértices em uma única lista.

- **Um hexágono** não é um vértice — são **seis**.
- Python já tem uma estrutura natural para agrupar vários valores: a **lista**.
- Cada elemento da lista é, por sua vez, outra lista com três números (x, y, z).



Como o Python faz isso?

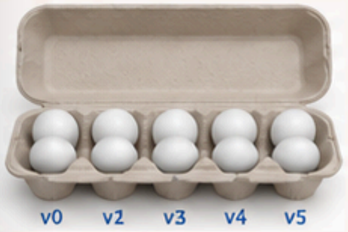
test-2-3.py

```
positionData = [
    [   0.8,   0.0,   0.0],
    [   0.4,   0.6,   0.0],
    [  -0.4,   0.6,   0.0],
    [  -0.8,   0.0,   0.0],
    [  -0.4,  -0.6,   0.0],
    [   0.4,  -0.6,   0.0]
]
```

Lista principal:
é uma coleção de 6 vértices.
(cada item é outro valor)

Cada sublista:
representa um vértice com três coordenadas (x, y, z).

Analogia: guardando pontos em uma caixa



Imagine uma caixa de ovos com 6 divisórias.
Cada bolinha é um vértice.
A caixa inteira é a lista.
Cada divisória guarda um conjunto de três valores (x, y, z).

Listas permitem organizar, acessar e enviar vários dados para a GPU de forma eficiente.

CPU e GPU

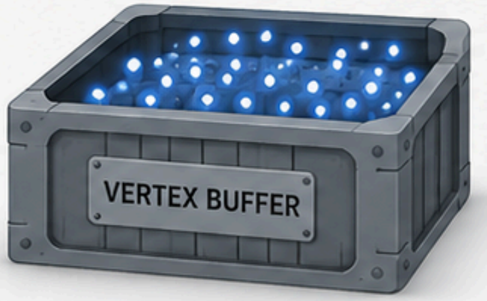
Python (CPU)

Organiza os vértices
(coordenadas, cores, etc.)

x	y	z	r	g	b
-0.5	0.8	0.0	1.0	0.5	0.2
0.6	0.2	0.0	0.2	1.0	0.4
-0.3	-0.6	0.0	0.2	0.7	1.0
0.7	-0.7	0.0	1.0	1.0	0.2
...	...	...	...	...	...

Vertex Buffer

Memória da GPU que armazena
todos os vértices de uma vez.



GPU

Usa os vértices do vertex buffer
para desenhar a imagem na tela.

- O código Python roda na CPU; quem desenha de fato é a GPU.
- Os dados dos vértices precisam sair da CPU e chegar até a GPU.
- Não queremos reenviar cada ponto manualmente, um por um, a cada quadro renderizado.
- A solução é enviar todos os dados de uma vez para um espaço de memória da GPU: o *vertex buffer*.

Analogia: a CPU separa todos os pontos, o *vertex buffer* guarda tudo em uma única caixa, e a GPU usa essa caixa para desenhar sem precisar pedir ponto por ponto.

CPU separa — Vertex Buffer guarda — GPU desenha

O que é um Vertex Buffer?

Um Vertex Buffer armazena os dados dos vértices para que a GPU possa utilizá-los durante a renderização.

1 CPU / Python

Organiza os vértices

```
# Vértices do polígono
vertices = [
    (0.0,  0.5),
    (0.5,  1.0),
    (1.0,  0.5),
    (0.8, -0.2),
    (0.2, -0.2)
]
```

Lista de vértices

1	(0.0, 0.5)
2	(0.5, 1.0)
3	(1.0, 0.5)
4	(0.8, -0.2)
5	(0.2, -0.2)
...	

2 Vertex Buffer (VBO)

Guarda tudo de uma vez

3 GPU

Desenha na tela

Analogia: a CPU junta todos os pontos em uma caixa.

Essa caixa é o Vertez Buffer.

A GPU abre essa caixa e usa os dados para desenhar.

CPU separa e organiza → *Vertex Buffer guarda* → GPU desenha

VBO = *caixa de dados da GPU*

Como o exemplo simplifica isso?

test-2-3.py

```
from core.attribute import Attribute
```

Criar buffers, enviar dados e associar variáveis manualmente exige várias funções OpenGL, em uma ordem específica.

Encapsulamos esse trabalho repetitivo em uma classe: Attribute.

A partir de agora, criar um vertex buffer bem configurado será tarefa de poucas linhas.

Attribute

esconde:

1. **criar o buffer**
   Gera o buffer na GPU e reserva memória.
2. **enviar os dados**
   Move os dados dos vértices da CPU para a memória da GPU.
3. **associar a variável do shader**
   Informa ao shader onde buscar cada dado do vértice.

Resultado: menos código, menos erros e mais foco naquilo que realmente importa: desenhar.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

Visão geral

A classe Attribute cuida de todo o ciclo dos dados:

1 Armazena dados na CPU

Recebe os dados e o tipo dos elementos.

2 Envia para a GPU (Vertex Buffer)

Cria um buffer na GPU e copia os dados.

3 Associa ao shader

Liga o buffer a uma variável do programa.

4 GPU usa durante a renderização

A GPU lê os dados diretamente do buffer.

Estrutura da classe

```
class Attribute(object):
```

- **dataType** — Tipo dos elementos: int | float | vec2 | vec3 | vec4
- **data** — Array com os dados dos vértices
- **bufferRef** — Referência do buffer criado na GPU

- `__init__(dataType, data)`
  Cria o buffer e envia os dados imediatamente
- `uploadData()`
  Converte e envia os dados para o buffer da GPU
- `AssociateVariable(programRef, variableName)`
  Associa o buffer à variável do shader

Como funciona (por dentro)

1. Inicialização (__init__)

1. Recebe dataType e data
2. Gera um buffer na GPU:
   `bufferRef = glGenBuffers(1)`
3. Chama uploadData()

```
Attribute("vec3", vertices)
```

2. Envio dos dados (uploadData)

1. Converte para numpy float32

0.8	0.0	0.0	0.4	0.6	0.0	...

float32

2. Seleciona o buffer

```
glBindBuffer(GL_ARRAY_BUFFER, bufferRef)
```

3. Copia os dados para a GPU

```
glBufferData(GL_ARRAY_BUFFER, data, GL_STATIC_DRAW)
```

Memória da GPU (Vertex Buffer)

0	0.8
1	0.0
2	0.0
3	0.4
4	0.6
5	0.0
...	...

(float32)

3. Associação ao shader (associateVariable)

1. Obtém a localização da variável no programa

```
glGetAttribLocation(programRef, variableName)
```

2. Seleciona o buffer

```
glBindBuffer(GL_ARRAY_BUFFER, bufferRef)
```

3. Informa como ler os dados

```
glVertexAttribPointer(...)
(1, 2, 3 ou 4 componentes)
```

4. Habilita o atributo

```
glEnableVertexAttribArray(variableRef)
```

Programa (Shader)

```
in vec3 position;
in vec3 color;
...
```

Ligação

position

color

...

VBO (Vertex Buffer)

0	0.8
1	0.0
2	0.0
3	0.4
4	0.6
5	0.0
...	...

Resumo

A classe Attribute automatiza as etaapas repetitivas do OpenGL:

- [x] Cria o buffer na GPU
- [x] Envia todos os dados de uma vez
- [x] Associa ao shader corretamente
- [x] Permite que a GPU leia os dados durante a renderização

Benefícios

- Menos código
- Menos erros
- Mais organização
- Mais desempenho

```
# Antes (várias chamadas OpenGL)
...
# Agora (poucas linhas)
attr = Attribute("vec3", vertices)
attr.associateVariable(program, "position")
```

Nosso programa: test-2-3.py

Visão geral da arquitetura

Arquitetura baseada em herança:
a classe **Test (Base)** organiza o ciclo do programa.

```
class Test(Base):
# Gerencia o ciclo da aplicação
```

initialize()

configura tudo uma única vez

- **shaders**
  Carrega e compila os vertex/fragment shaders.
- **program**
  Cria e vincula o programa (shader program).
- **VAO**
  Cria o Vertex Array Object e configura.
- **vertex data**
  Envia os dados dos vértices (via Attribute).

update()

roda a cada quadro (~60x/s)

- **glUseProgram**
  Ativa o programa (shaders) para ser usado na GPU.
- **glDrawArrays**
  Desenha os vértices (triângulos/linhas/pontos) usando os dados do VAO.

Python (CPU) → configura e envia → VAO + VBO (memória da GPU) → usado a cada quadro → GPU renderiza

Resultado:
Inicializa uma vez, atualiza sempre.

eficiente e rápido!

Código: Imports

test-2-3.py

```python
from core.base import Base
from core.openGLUtils import OpenGLUtils
from core.attribute import Attribute
from OpenGL.GL import *
```

Import	Responsabilidade
Base	estrutura da aplicação (janela, laço de eventos)
OpenGLUtils	compilar e ligar os shaders (o programa da GPU)
Attribute	guardar e enviar dados dos vértices
OpenGL.GL	acesso às funções e constantes do OpenGL

A classe Test

test-2-3.py

```
class Test(Base):
    def initialize(self):
        ...

    def update(self):
        ...
```

initialize() → configura

prepara shaders, VAO e dados

update() → desenha continuamente

executa a cada quadro

A classe **Test** separa o que é configuração inicial do que é desenho contínuo.

Vertex Shader

Pequeno programa que roda na GPU para cada vértice e decide onde ele aparecerá na tela.

vsCode (GLSL)

```
1   in vec3 position;
2
3   void main()
4   {
5       gl_Position = vec4(
6           position.x,
7           position.y,
8           position.z,
9           1.0
10      );
11  }
```

Entrada: position (vec3)

Recebe as coordenadas (x, y, z) de cada vértice vindas de um buffer (VBO).

Saída: gl_Position (vec4)

Define a posição final do vértice na tela (x, y, z, w). O valor 1.0 em w é o padrão para posições.

Executado por vértice

Este shader é executado UMA VEZ PARA CADA um dos seis vértices do nosso objeto.

De onde vem?

Os vértices são armazenados na memória da GPU (VBO).

Exemplo de 3 vértices (vec3)

- v0: (-0.5, -0.5, 0.0)
- v1: (0.5, -0.5, 0.0)
- v2: (0.0, 0.5, 0.0)

...

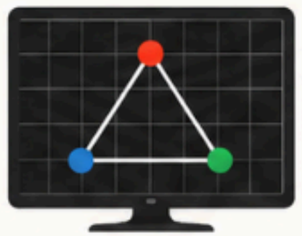
GPU Pipeline do Vertex Shader

Vértice 0	→	Vertex Shader	→ gl_Position
Vértice 1	→	Vertex Shader	→ gl_Position
Vértice 2	→	Vertex Shader	→ gl_Position

...

Resultado

As posições calculadas são usadas para montar as formas e desenhar na tela.



Em resumo

Este shader apenas pega a posição que já recebeu e a repassa para o pipeline gráfico, dizendo onde cada vértice deve ficar na tela.

O que significa in vec3 position?

`in`

↓

é um dado de entrada do shader
que vem de um buffer

`vec3`

↓

significa que o valor tem
3 componentes: x, y e z

`position`

↓

é o nome da variável
que vai guardar essa posição

`position` vai receber, a cada execução, um vetor **`[x, y, z]`** extraído de positionData.

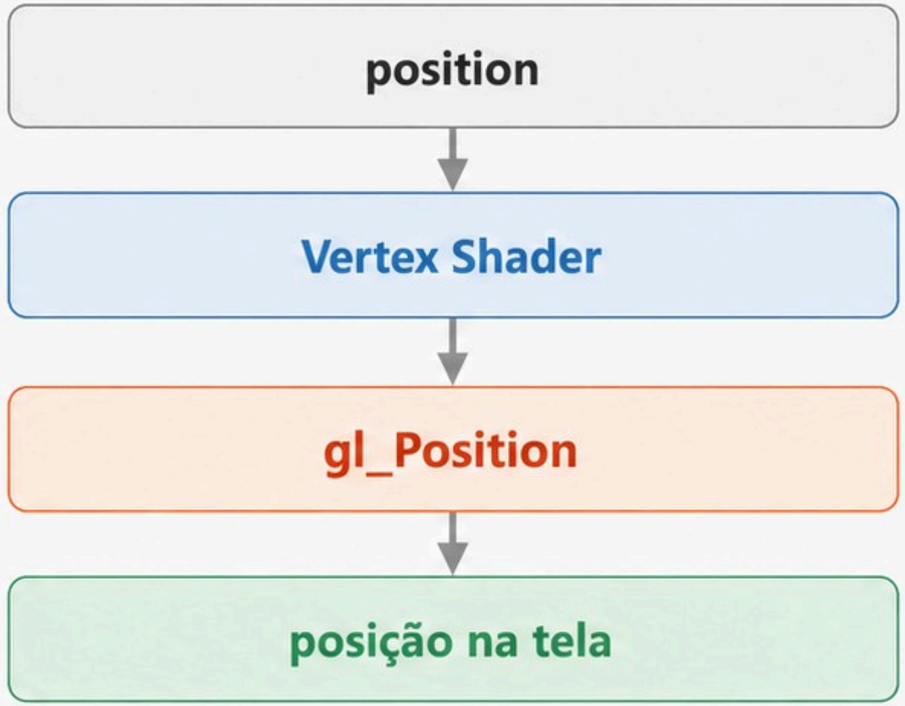
Em outras palavras: o shader pega uma coordenada por vez e guarda essa posição na variável **position**.

gl_Position

vS Code

```
gl_Position = vec4(...);
```

- gl_Position é uma variável especial e obrigatória — o OpenGL já sabe o que fazer com ela.
- O trabalho do vertex shader é colocar nessa variável um valor vec4 válido, indicando onde o vértice deve aparecer.



Em resumo: o shader recebe a posição, grava o resultado em **gl_Position** e o OpenGL usa isso para colocar o vértice na tela.

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Fragment Shader

Define a cor final de cada ponto desenhado na tela

É um pequeno programa da GPU executado para cada fragmento (cada pedacinho da imagem que será pintado). Sua função é decidir a cor final que aparecerá na tela.

fsCode (GLSL)

```
out vec4 fragColor;

void main()
{
    fragColor = vec4(
        1.0,
        1.0,
        0.0,
        1.0
    );
}
```

out vec4 fragColor — é a variável de saída onde o shader escreve a cor final.

vec4 — guarda 4 valores: **R, G, B** e **A**.

R, G, B — definem a mistura de vermelho, verde e azul.

A — é a transparência (alpha).
1.0 = totalmente visível.

Valores RGBA deste exemplo

R	=	1.0
G	=	1.0
B	=	0.0
A	=	1.0

Resultado → AMARELO

Fragmento
Um pedacinho da imagem a ser pintado.

→

Fragment Shader
O shader decide a cor final para esse fragmento.

→

Cor final na tela
A cor calculada é aplicada e aparece na imagem.

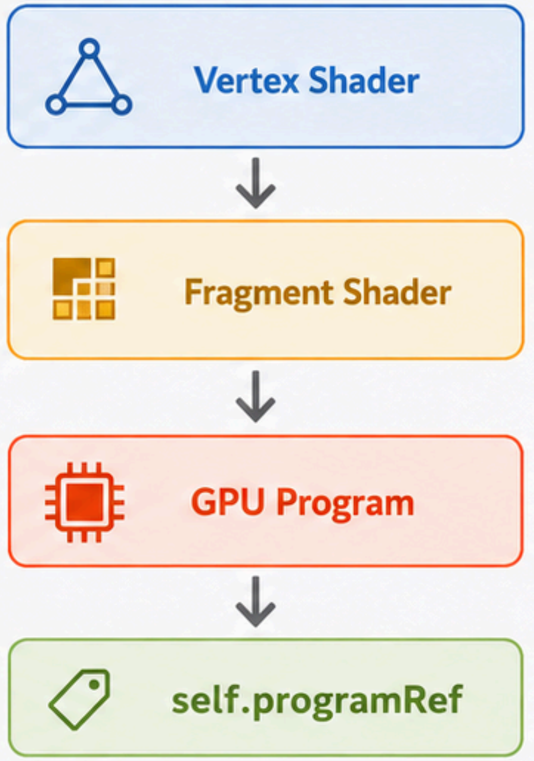
Em resumo: o fragment shader escolhe a cor de cada pedacinho desenhado. Neste exemplo, ele pinta tudo de amarelo.

Criando o programa da GPU

test-2-3.py

```
self.programRef = OpenGLUtils.initializeProgram(
    vsCode,
    fsCode
)
```

- **vsCode** contém o código do **Vertex Shader**.
- **fsCode** contém o código do **Fragment Shader**.
- **OpenGLUtils.initializeProgram(...)** compila os dois shaders e liga os dois em um único programa da GPU.
- Esse programa é o pacote que a GPU vai usar para desenhar.



Por que self.programRef?

- "programRef" é a referência/ identificador do programa criado na GPU.
- "self" guarda essa referência dentro do objeto da classe.
- Assim, o programa pode ser reutilizado depois, por exemplo em **glUseProgram(self.programRef)**.
- Sem guardar essa referência, o código não saberia qual programa da GPU deve ativar.

Em resumo: o código junta o **Vertex Shader** e o **Fragment Shader** em um programa da GPU e guarda sua referência em **self.programRef** para usar esse programa depois.

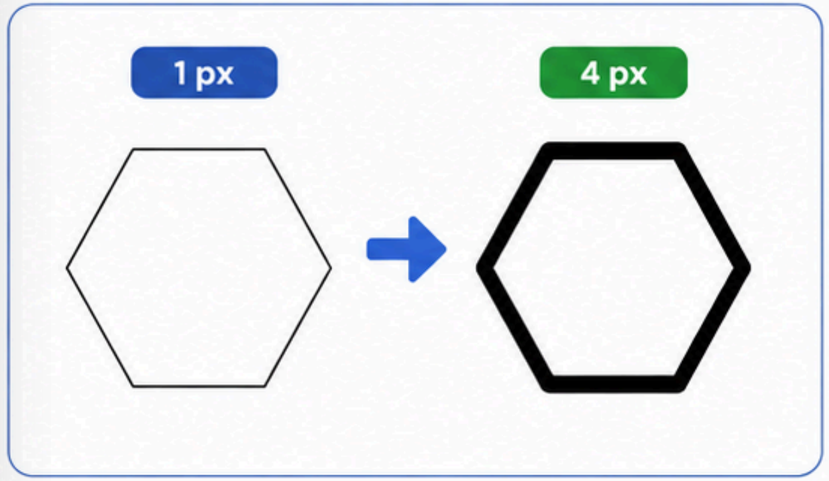
unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

Espessura da linha: glLineWidth()

```
glLineWidth(4)
```

Comparação da espessura



O que é?
Uma função que define a espessura das linhas desenhadas.

Para que serve?
Facilitar a visualização do contorno do hexágono.

O que recebe?
Um número inteiro: a largura em pixels.

O que muda?
As linhas passam de **1 pixel** para **4 pixels** de espessura.

Resultado
Um hexágono com contorno bem visível na tela.

Resultado na prática

Hexágono com contorno destacado, facilitando a percepção da forma e das arestas.

Vertex Array Object

test-2-3.py

```
vaoRef = glGenVertexArrays(1)
glBindVertexArray(vaoRef)
```

O VAO guarda a configuração necessária para o OpenGL interpretar os atributos de vértice.

O VAO não armazena os dados dos vértices.
Ele lembra ao OpenGL:
"onde estão os dados e como cada atributo deve ser lido".

VBO
(Vertex Buffer Object)

ARMAZENA OS DADOS DOS VÉRTICES

É onde os vértices realmente vivem na memória da GPU.
Contém posições, cores, normais, coordenadas de textura...

EXEMPLO: posições (x, y, z)

-0.5	-0.5	0.0	0.5	-0.5	0.0	0.0	0.5	0.0	...
x	y	z	x	y	z	x	y	z	

É o CONTEÚDO (OS DADOS).
Sem VBO não há dados para desenhar.

DADOS
(no VBO)

CONFIGURAÇÃO
(no VAO)

VAO
(Vertex Array Object)

ARMAZENA A CONFIGURAÇÃO DE COMO LER OS DADOS

Informa ao OpenGL **como interpretar** os dados que estão no VBO.
O VAO registra:

- Qual VBO está sendo usado
- Qual atributo está habilitado (ex.: posição, cor, normal...)
- Número de componentes (ex.: 3 para posição x, y, z)
- Tipo de dado (ex.: GL_FLOAT)
- Tamanho do stride (passo entre vértices)
- Offset (deslocamento inicial dentro do VBO)

É a RECEITA (A INTERPRETAÇÃO).
Sem VAO, o OpenGL não sabe como ler os dados do VBO.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

Os seis vértices

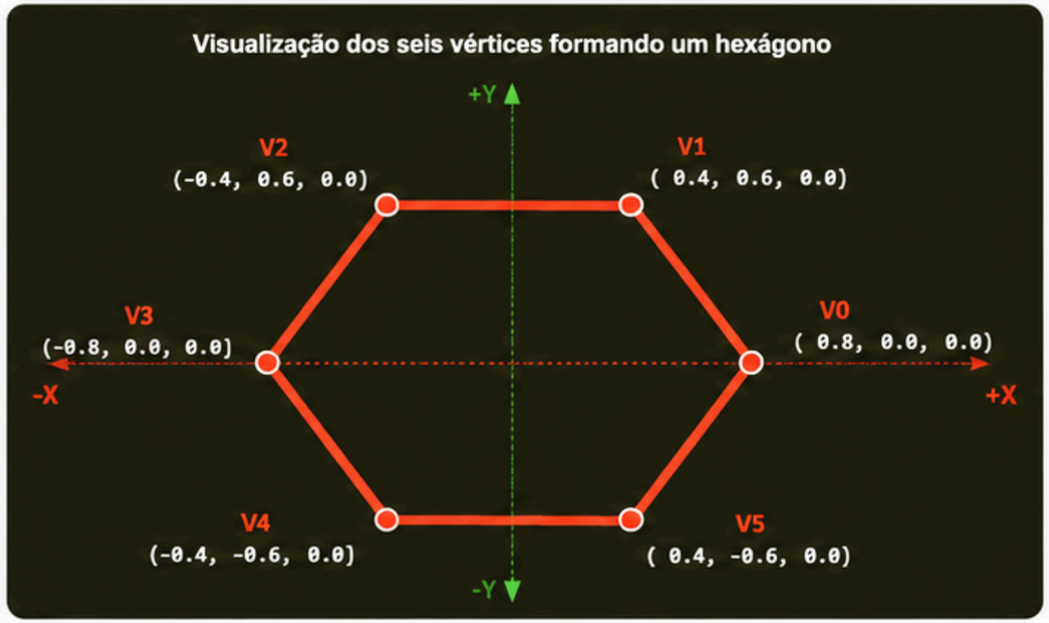
Cada vértice é definido por suas coordenadas (x, y, z) no espaço 3D.

Índice	Coordenadas (x, y, z)
V0	[0.8, 0.0, 0.0]
V1	[0.4, 0.6, 0.0]
V2	[-0.4, 0.6, 0.0]
V3	[-0.8, 0.0, 0.0]
V4	[-0.4, -0.6, 0.0]
V5	[0.4, -0.6, 0.0]

O que significam os números?

x → posição horizontal
y → posição vertical
z → profundidade

Todos os vértices estão no plano XY, pois z = 0.0 para todos eles.



O que é um vértice?

Um vértice é um ponto no espaço 3D usado para construir formas geométricas. Neste exemplo, temos **6 vértices** que, conectados em ordem, formam um **hexágono**.

Sistema de coordenadas

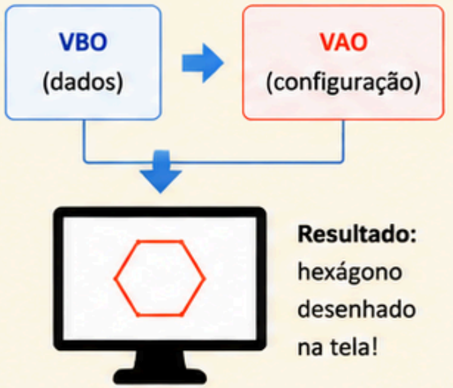
- Origem (0, 0, 0) no centro.
- +X → direita | -X → esquerda
- +Y → cima | -Y → baixo
- +Z → saindo da tela
- -Z → entrando na tela

Ordem de conexão dos vértices (para formar o hexágono)

V0 → V1 → V2 → V3 → V4 → V5 → V0 (e fecha o polígono)

Uso prático

Esses vértices são enviados para a GPU (no VBO) e, com a configuração correta (no VAO), o OpenGL usa essas posições para desenhar o hexágono na tela.



Resultado: hexágono desenhado na tela!

Quantos vértices temos?

```
self.vertexCount = len(positionData)
```

vertexCount = 6

Pergunta aos alunos: por que não escrevemos simplesmente 6?

Porque o programa se adapta automaticamente ao tamanho da lista.

Se amanhã **positionData** tiver 8 pontos, **vertexCount** também mudará **sem precisar editar** essa linha.

positionData (lista de pontos) → **len( )** Conta quantos elementos há na lista. → **vertexCount** (número de vértices) **6**

Atualização automática!

Hoje: 6 pontos → vertexCount = 6

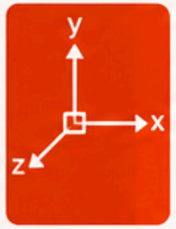
Amanhã: 8 pontos → vertexCount = 8

Criando o atributo position

Definindo o tipo e enviando as posições dos vértices para a GPU

test-2-3.py

```
positionAttribute = Attribute(
    "vec3",
    positionData
)
```



"vec3"

Define o tipo do atributo. Cada vértice possui 3 componentes: x, y e z.

- x = posição horizontal
- y = posição vertical
- z = profundidade
- neste exemplo, cada vértice é representado por 3 números

positionData

É a lista com os dados dos 6 vértices do hexágono.

- contém as coordenadas de todos os pontos
- cada item da lista representa um vértice
- esses dados serão enviados para a GPU
- o OpenGL usará esses valores para desenhar a forma

Attribute(...)

Cria um objeto da classe Attribute para preparar esse atributo para o OpenGL.

- guarda o tipo e os dados
- cria um buffer na GPU
- envia os dados automaticamente para esse buffer
- depois, esse atributo poderá ser associado ao shader

Ligando Python ao Vertex Shader

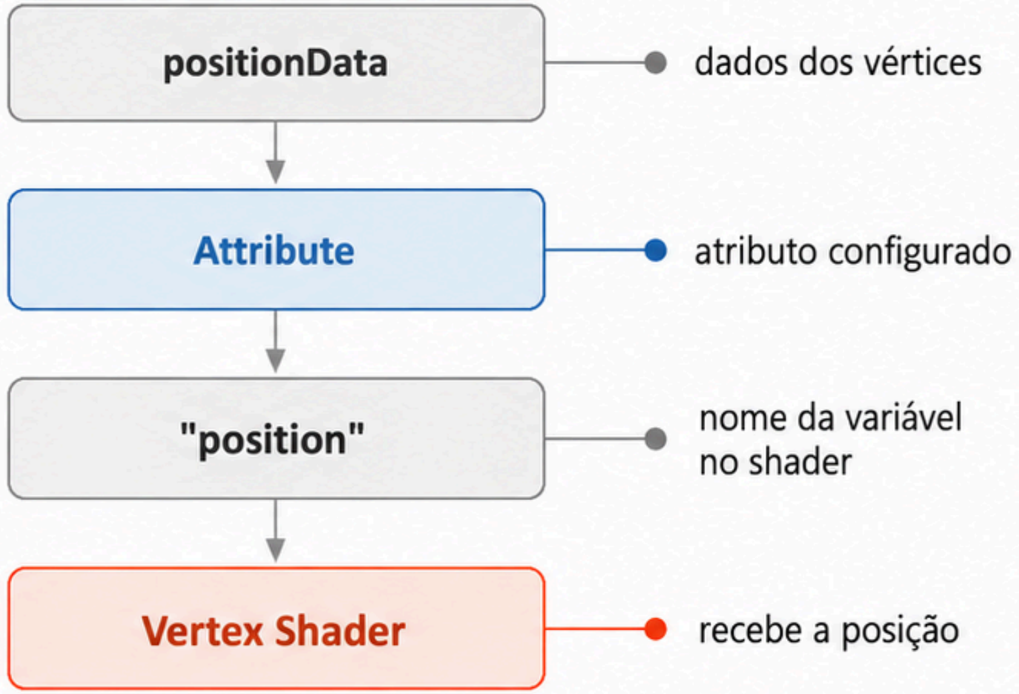
test=2=3.py

```
positionAttribute.associateVariable(
    self.programRef,
    "position"
)
```

associateVariable(...) faz a ligação entre os dados preparados no Python e a variável do shader.

self.programRef é a referência do programa da GPU já criado com os shaders compilados e ligados.

"position" é o nome da variável declarada no vertex shader.



Em resumo: o **Python prepara os dados,** a classe **Attribute** organiza essa ligação, e o **Vertex Shader** passa a receber cada posição do buffer.

unipac.br
Barbacena

C. COMPUTAÇÃO UNIPAC

Pipeline Gráfica: do código ao pixel na tela

Caminho percorrido pelos vértices e fragmentos até formarem a imagem final.

1. Entrada de Dados
Vértices no VBO (posições, cores, normais, etc.)

2. Vertex Shader
Transforma vértices (modelo → mundo → visão → projeção)

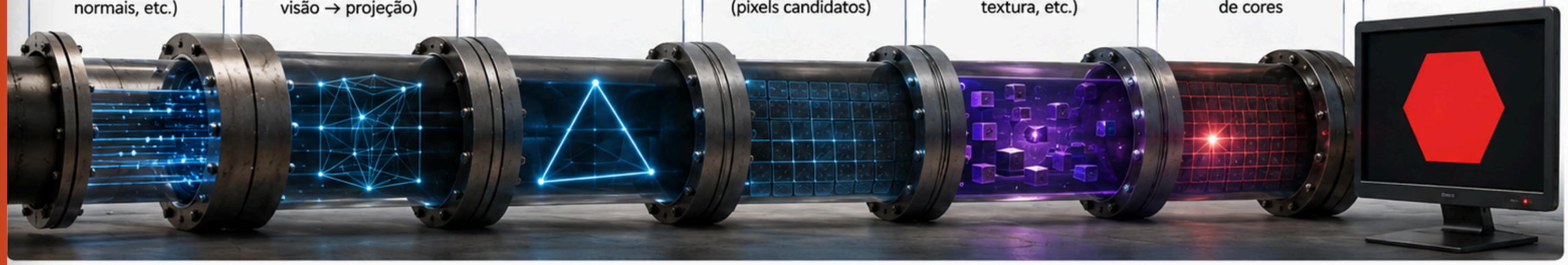
3. Primitive Assembly
Forma primitivas (triângulos, linhas, etc.)

4. Rasterização
Converte primitivas em fragmentos (pixels candidatos)

5. Fragment Shader
Calcula cor de cada fragmento (iluminação, textura, etc.)

6. Testes e Mistura
Testes (profundidade, stencil) e blending de cores

7. Saída
Pixels finais na tela

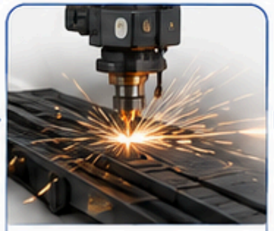


Analogia: fábrica de produção

1. Matéria-prima
Chegam os materiais brutos (vértices).

2. Modelagem
As peças são moldadas e transformadas.

3. Montagem
As peças são montadas em produtos (primitivas).



4. Corte/Preparação
Os produtos são divididos em partes menores (fragmentos).

5. Pintura/Acabamento
Cada parte recebe cor, textura e detalhes.

6. Inspeção/Embalagem
Verificação de qualidade e preparação para entrega (testes e mistura).

7. Produto Final
Produto pronto entregue ao cliente (pixels na tela).

update()

Função responsável por atualizar os estados e desenhar os vértices na tela a cada quadro (frame).

test-2-3.py

```python
def update(self):
    glUseProgram(self.programRef)
    glDrawArrays(
        GL_LINE_LOOP,
        0,
        self.vertexCount
    )
```

Explicação do código

1. **def update(self):**
   Define o método *update*, que será chamado a cada ciclo de atualização do programa (geralmente a **cada quadro** da aplicação).

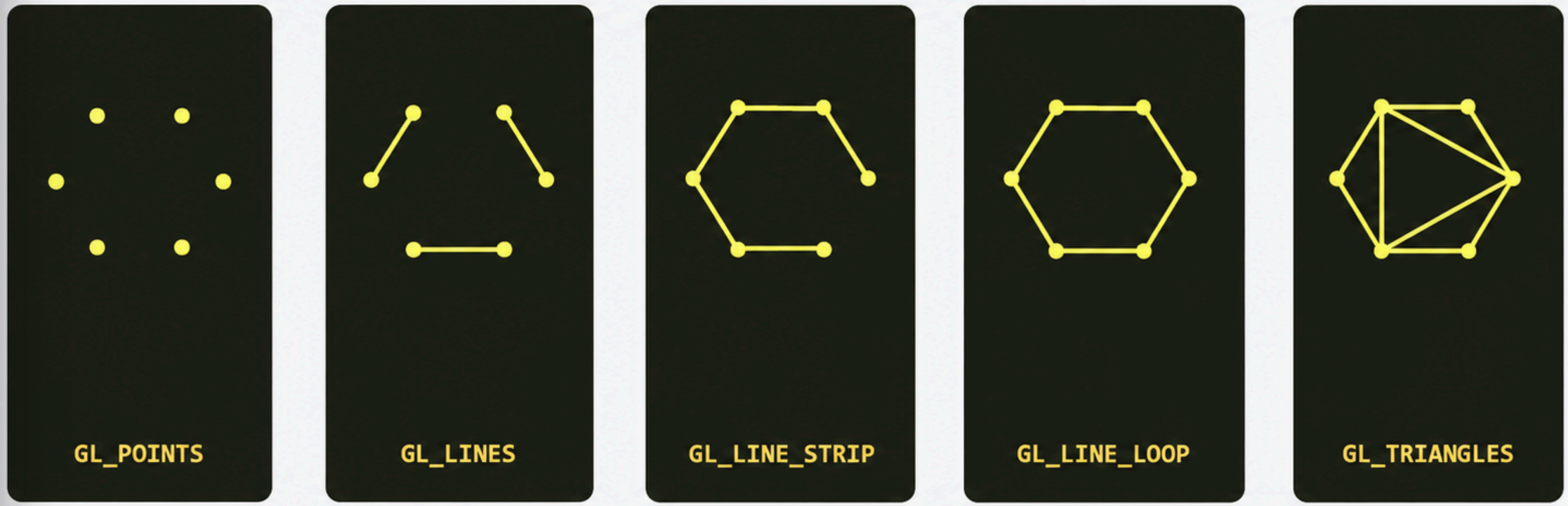
2. **glUseProgram(self.programRef)**
   Ativa (torna **atual**) o **programa de shader** armazenado em *self.programRef*. Isso informa ao OpenGL quais shaders de vértice e fragmento serão usados para renderizar os objetos.

3. **glDrawArrays(GL_LINE_LOOP, 0, self.vertexCount)**
   Comando que realiza o **desenho** dos vértices.
   - GL_LINE_LOOP: modo de desenho que conecta os vértices em ordem e fecha a forma ligando o último vértice ao primeiro. Ideal para desenhar contornos (como o hexágono).
   - 0: **índice inicial** do primeiro vértice a ser desenhado no **array**.
   - self.vertexCount: quantidade total de vértices a serem desenhados. No nosso caso, é 6 (os seis vértices do hexágono).

Resumo
A função *update* ativa o shader e desenha a figura usando todos os vértices disponíveis, conectando-os em loop para formar o polígono.

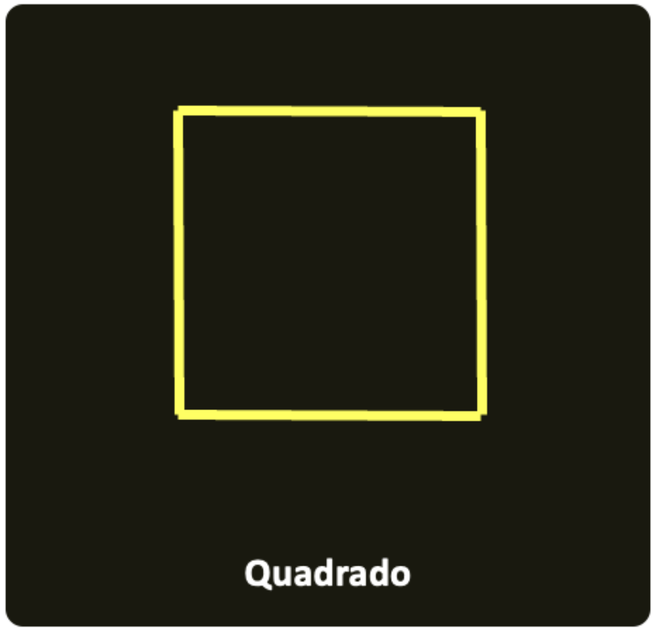
Mudando a primitiva

Os mesmos seis vértices, resultados diferentes



A primitiva informa ao OpenGL *como os vértices devem ser conectados.*

Do hexágono para outras formas



A lógica permanece a mesma. O que muda são os vértices e a primitiva.

unipac.br
Barbacena

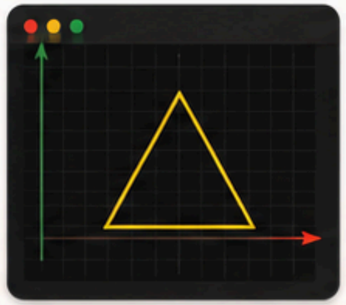
C. COMPUTAÇÃO UNIPAC

Agora é sua vez

Modifique o test-2-3.py

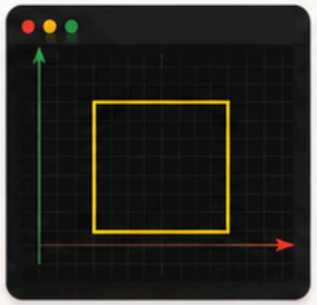
1 Desafio 1

Transformar o hexágono em um triângulo.



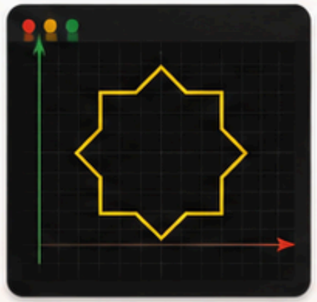
2 Desafio 2

Transformar em um quadrado.



3 Desafio 3

Criar uma forma própria.



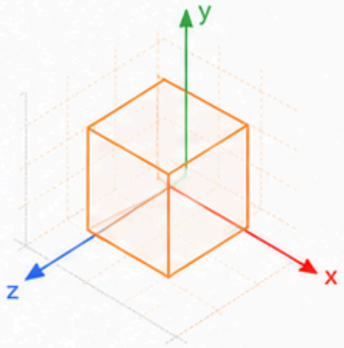
O que alterar

```
1  positionData
2  glDrawArrays(...)
3  # quando necessário
4  # Linha, primitiva ou quantidade de vértices
5  # podem ser alteradas quando necessário.
```

Dica: pense em vértices, primitiva, shader e resultado visual na tela.

Perguntas para responder

1. Quantos vértices sua forma possui?
2. Quais coordenadas foram utilizadas?
3. Qual primitiva foi escolhida?
4. Qual é a função de Attribute?
5. Qual é a função do Vertex Shader?
6. O que glDrawArrays() faz?



CONTATO

rodrigofernandes@unipac.br

OBRIGADO!!